The Problem:

Buying anything online **needs a lot of research** to get the best worth for money. This consumes a **lot of time**. With the vast variety of products available online and e-commerce websites, many a times a lot of cross checking is involved. For certain technical products, specifications also need to be compared. Sometimes certain additional constraints (like delivery time) also need to be met. While one can ask a general-purpose AI to compare items, this may not work well since the AI doesn't have access to all the specific information and explaining it the context again wastes time. This is the motivation for creating a fast, easy-to-use E-commerce assistant which does the research job for you.

The Solution:

An **LLM based e-commerce analysis agent** which performs all the research about the products across various websites and their credibility and shortlists a bunch of **budget-friendly**, **fast-delivery good-quality** trustable candidates, out of which the user can choose what they want, with further conversation with the LLM if required. The current focus is primarily on online shopping but, if possible, it will be **extended to food delivery, ride-booking, grocery delivery etc.**

User Interface:

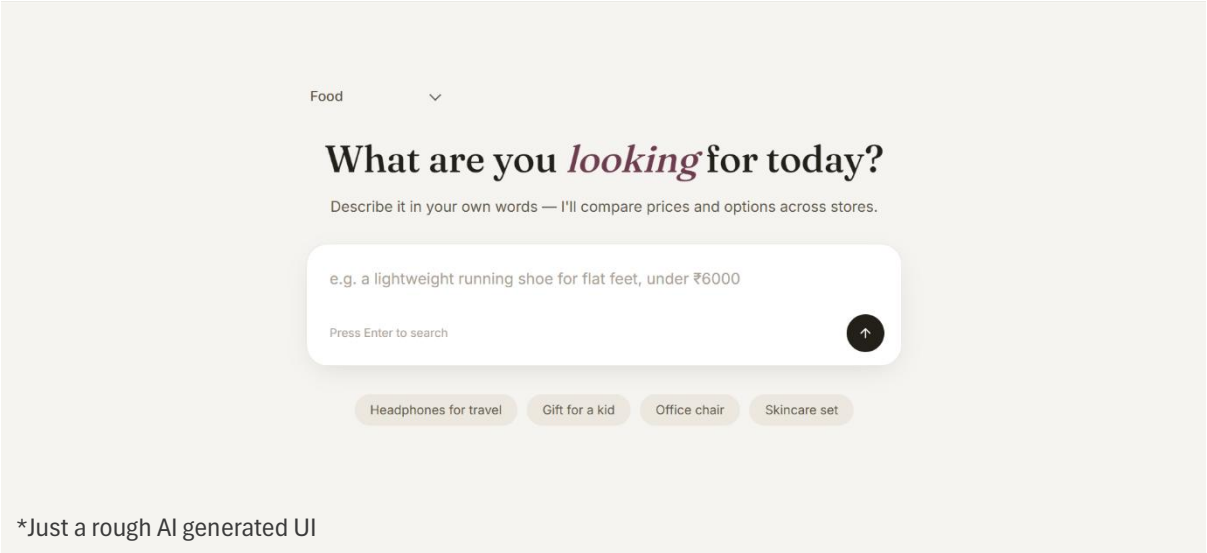
The user will use it through a web application or a mobile app.

- The user opens the application and selects the category of service (online shopping, food delivery, transportation, etc.) each of which is associated with a set of possible apps (the user can add or remove apps to this based on their preference).
- Then the user describes what they want, with all the necessary details. They can select certain filters (like a slider for budget).
- The AI then reads all the info and does all the research it wants to and outputs a bunch of assorted options which are best for the user, along with short descriptions.
- The user can further ask the AI for recommendations or questions.
- The user then clicks the option they like, and it redirects them to the respective site/application where they can complete the purchase.
- Later, a Google Chrome extension can also be made for users to easily compare a bunch of selected products.

Existing solutions:

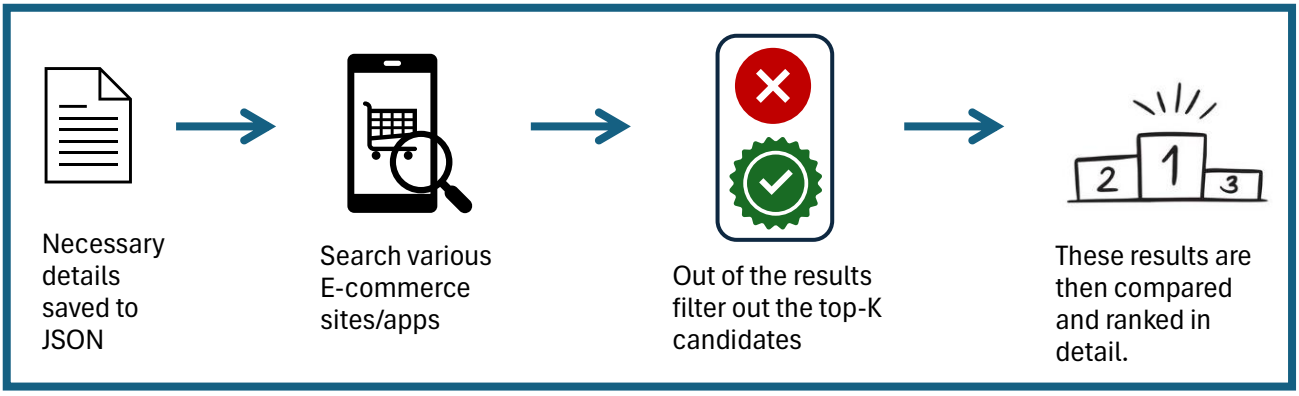
Since it's not a very unique idea, **some solutions already exist.** There are some apps that allow the user to compare products across different platforms, but they **do not use any AI** to do the research. There are some AI based tools which make comparisons, but there are a few shortcomings. Firstly, these tools do it **only for online shopping**, as opposed to the versatility aimed in this project. They are **mostly American based**, so they don't check Indian E-commerce sites. They use a **server side LLM and not a local one**, which may cause privacy concerns.

Implementation plan:



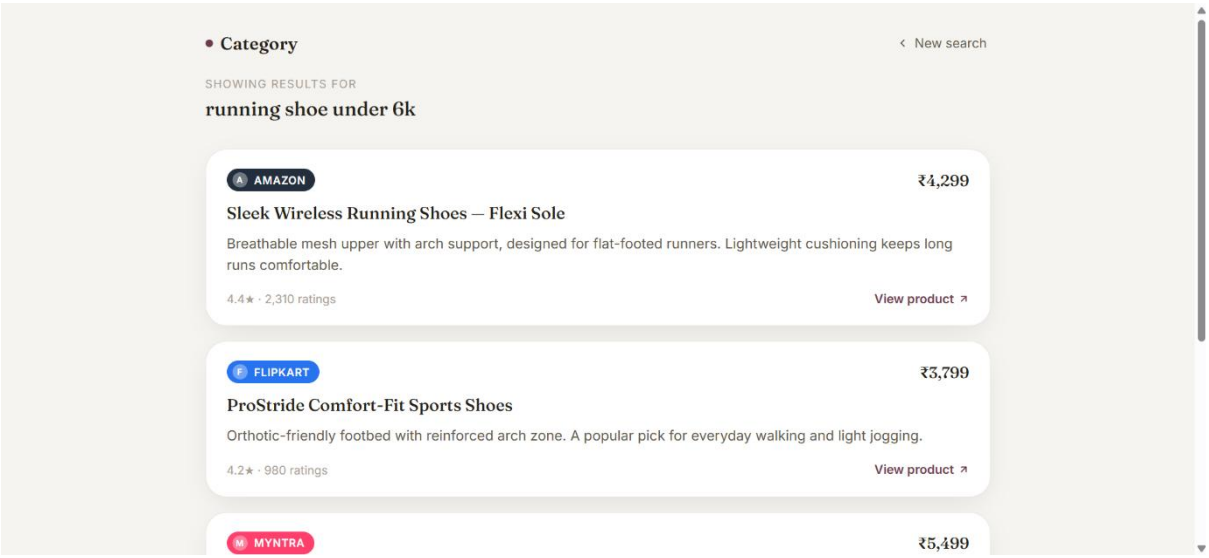
*Just a rough AI generated UI

User selects a category and describes his needs in the text box



Backend processing

The results are ordered with the most fitting deal at the top.



*Just a rough AI generated UI

- As mentioned earlier, the user input will be through a web/local app. Through various input formats, it takes in the **necessary information and saves it** to a local JSON file and calls the python backend.
- The backend will call the client-side LLM giving it the primary inputs and asking it to **generate a search query for the E-commerce sites/apps** and a list of filters.
- This query is used to search for the possible products in the set of service providers mentioned in the category. The products are **filtered either using the filters provided by the app or by using an automated script.**
- The surviving products are then presented to the LLM which **rates each product using a lightweight prompt** on the required criteria, and a weighted average of these ratings is considered as the **overall rough score**. Then the **top-K products are selected** based on these scores.
- After this rough round, the LLM then **analyzes these top-K options in detail**, comparing all relevant specifications by generating **head-to-head comparison table** for all the products and it ranks them using a weighted Pareto ranking system. It also analyzes the customer reviews and other aspects of the products.
- The LLM then generates a small description for these ranked products explaining the pros and cons.
- LLM model used will be either **Qwen-3.5-2B** or **Gemma-4-E2B**, focusing on mainly speed (for reading so much data) and good reasoning skills (to generate credible scores).
- All this output is **saved as an XML** and after finishing, the backend returns back to the frontend which parses the XML and neatly displays the data.
- Any **further questions** are directly relayed to the LLM with the required context.
- Each option is **linked to the respective site/app** where it can be purchased.
- In case of a chrome extension, the ranking system is called directly on the user selected products.

Opportunities for optimization (not much idea about this):

- Fine-tuned LLM for comparison. It can also be fine-tuned per service.
- GPU optimizations to read many sites simultaneously.
- Simultaneous processing of mutliple LLM prompts.
- Specialized memory structures for storing and comparing data.

Challenges:

- This will need a HUGE number of tokens (mainly for reading info etc.). And given that the speed of a normal LLM on a local GPU will be barely around 50 tokens/sec, it **might take very long** to process. Moreover, the **LLM needs to be called many times**.
- Reading from a website is easy **but accessing data from apps is much harder**, for services like transport, food delivery etc. Certain APIs might be required, and they are not easy to obtain.
- **Robot protections** in the websites/apps can hinder information gathering.
- There's **no pre-existing data for fine-tuning**. Since the problems are mainly based on situations, it is hard to create/obtain data for training.